Programming with C++

COMP2011: Program Flow Control II

Gary Chan
James Cheung
Cindy Li

Department of Computer Science & Engineering
The Hong Kong University of Science and Technology
Hong Kong SAR, China



Part I

Nested Looooooops



Nested Loops Example: Compute Average Score

One may put a while loop inside another while loop.

```
#include <iostream>                                          /* File: nested-while-avg.cpp */
using namespace std;

int main( )
{
    int NUM_ASSIGNMENTS = 5;                                 // Uppercase variable doesn't change
    int j;                                                   // Assignment counter
    int score, sum_of_scores;
    char reply = 'y';                                        // 'y' for yes, 'n' for no; initialized to yes

    cout << "Enter scores for the first student?  (y/n) " << endl;

    while ((cin >> reply) && (reply == 'y' || reply == 'Y'))
    {
        sum_of_scores = 0;                                   // Reset the accumulator to zero
        j = 1;                                               // Reset the assignment counter to 1

        while (j <= NUM_ASSIGNMENTS)
        {
            cout << "Enter student's score for assignment #" << j << " :  ";
            cin >> score;                                    // Remark: one should check input errors here
            sum_of_scores += score;
            j++;
        }

        cout << "The average score = " << sum_of_scores/NUM_ASSIGNMENTS << endl;
        cout << "Enter scores for another student?  (y/n) " ;
    }

    return 0;
}
```

Nested Loops Example: Multiplication Table

```
#include <iostream> /* File: multiplication-table.cpp */
include <iomanip> // a library that helps control input/output formats
using namespace std;

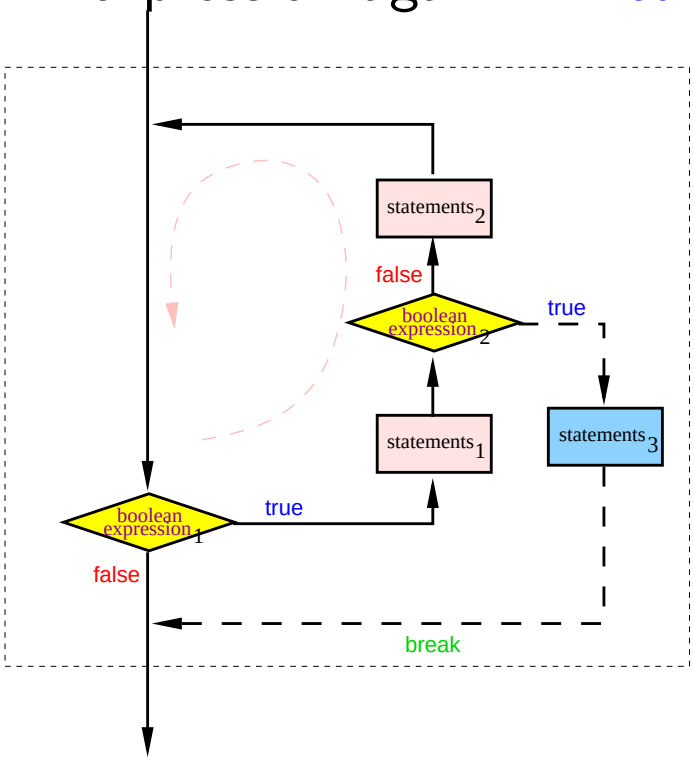
int main()
{
    // To print out products of j*k where j, k = 1,...,10
    for (int j = 1; j <= 10; ++j)
    {
        for (int k = 1; k <= 10; ++k) // Reset k=1 for each j. Why?
            cout << setw(4) << j*k;   // Set the length of output field to 4

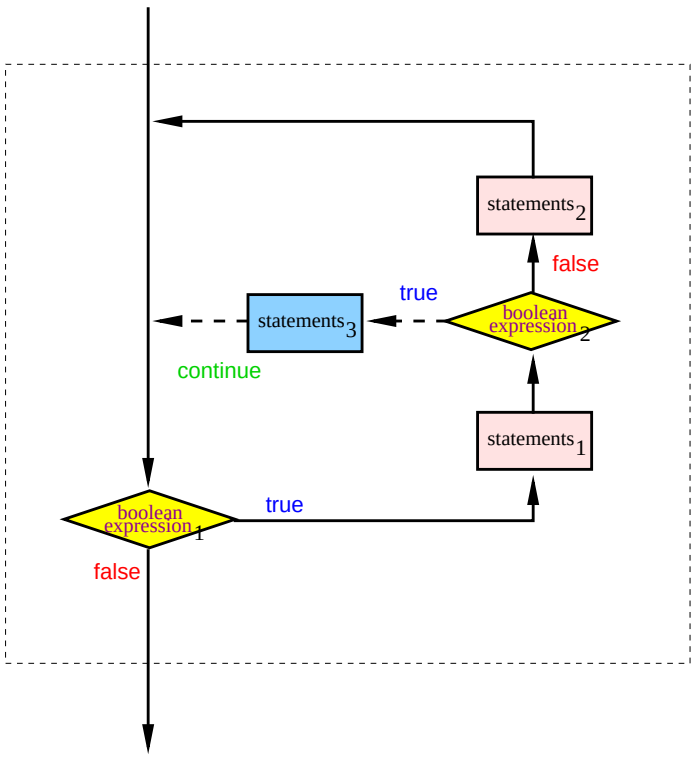
        cout << endl;
    }

    return 0;
}
```

break and continue

- A break causes the innermost enclosing loop to exit immediately.
- A continue causes the next iteration of the enclosing loop to begin.
- That is, in the while loop, control passes to test the boolean expression again immediately.





Example: Stop Inputs with break

```
#include <iostream> /* File: break-avg.cpp */
using namespace std;

int main()
{
    int NUM_ASSIGNMENTS = 5; // Uppercase variable doesn't change
    int j;                   // Assignment counter
    int score, sum_of_scores;
    char reply = 'y';        // 'y' for yes, 'n' for no; initialized to yes

    cout << "Enter scores for the first student? (y/n) " << endl;
    while ((cin >> reply)  &&  (reply == 'y' || reply == 'Y'))
    {
        sum_of_scores = 0;   // Reset the accumulator to zero
        j = 1;               // Reset the assignment counter to 1

        while (j <= NUM_ASSIGNMENTS)
        {
            cout << "Enter student's score for assignment #" << j << " : ";
            cin >> score;    // Remark: one should check input errors here

            if (score < 0)
                break;

            sum_of_scores += score;
            j++;
        }
        cout << "The average score = " << sum_of_scores/NUM_ASSIGNMENTS << endl;
        cout << "Enter scores for another student? (y/n) " ;
    }
    return 0;
} // Question: What is the output with the input: 4, 5, -6, 7, 8?
```

Example: Ignore Negative Inputs with continue

```
#include <iostream> /* File: continue-avg.cpp */
using namespace std;

int main()
{
    int NUM_ASSIGNMENTS = 5; // Uppercase variable doesn't change
    int j;                   // Assignment counter
    int score, sum_of_scores;
    char reply = 'y';        // 'y' for yes, 'n' for no; initialized to yes

    cout << "Enter scores for the first student? (y/n) " << endl;
    while ((cin >> reply)  &&  (reply == 'y' || reply == 'Y'))
    {
        sum_of_scores = 0;   // Reset the accumulator to zero
        j = 1;               // Reset the assignment counter to 1

        while (j <= NUM_ASSIGNMENTS)
        {
            cout << "Enter student's score for assignment #" << j << " : ";
            cin >> score;    // Remark: one should check input errors here

            if (score < 0)
                continue;

            sum_of_scores += score;
            j++;
        }
        cout << "The average score = " << sum_of_scores/NUM_ASSIGNMENTS << endl;
        cout << "Enter scores for another student? (y/n) " ;
    }
    return 0;
} // Question: What is the output with the input: 4, 5, -6, 7, 8 ?
```

Example: Difference between break and continue

```
/* File: break-example.cpp */
#include <iostream>
using namespace std;

int main()
{
    int j = 0;

    while (j < 3)
    {
        cout << "Enter iteration "
             << j << endl;

        if (j == 1)
            break;

        cout << "Leave iteration "
             << j << endl;
        j++;
    }

    return 0;
}
```

```
/* File: continue-example.cpp */
#include <iostream>
using namespace std;

int main()
{
    int j = 0;

    while (j < 3)
    {
        cout << "Enter iteration "
             << j << endl;

        if (j == 1)
            continue;

        cout << "Leave iteration "
             << j << endl;
        j++;
    }

    return 0;
}
```

Question: What are the outputs of the 2 programs?

Where Does continue; Continue in a for Loop?

```
#include <iostream> /* File: for-continue.cpp */
using namespace std;

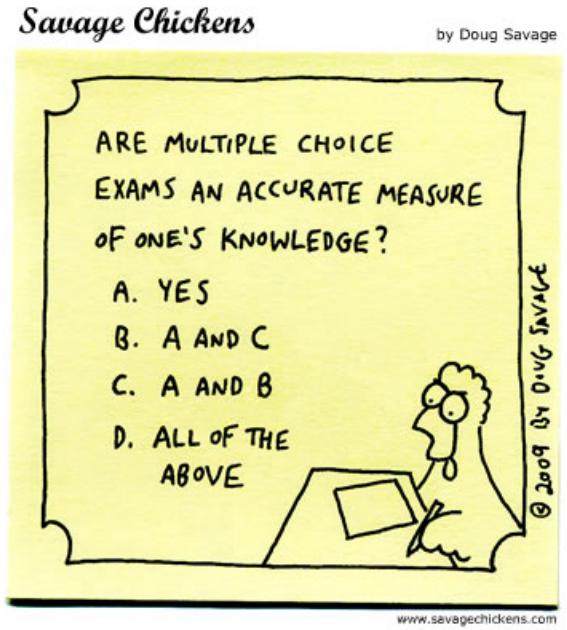
int main()
{
    for (int j = 1; j <= 10; j++)
    {
        cout << "j = " << j << endl;

        if (j == 3)
        {
            j = 10;
            continue;   // What if it is replaced by break;
        }
    }

    return 0;
}
```

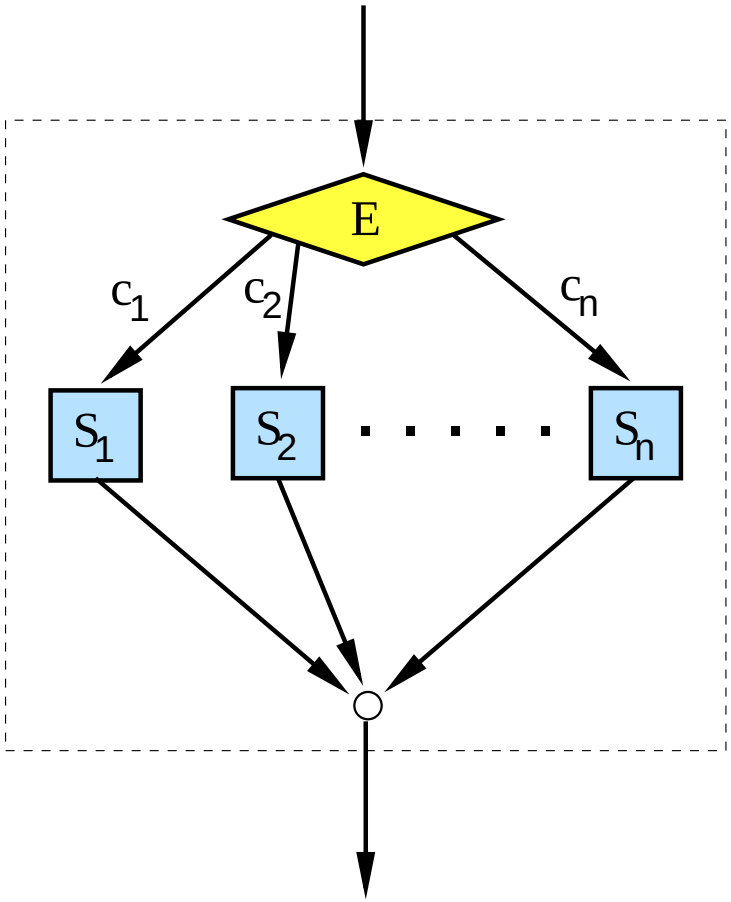
Part II

Let's **switch**: C++ Multiple Choices



switch Statement

switch statement is a variant of the if-else-if statement, that allows multiple choices based on the value of an integral expression.



Syntax: switch Statement

```
switch (integral expression)
{
    case constant-1:
        statement-sequence-1;
        break;
    case constant-2:
        statement-sequence-2;
        break;
    ...
    case constant-N:
        statement-sequence-N;
        break;
    default: // optional
        statement-sequence-(N+1);
}
```

Example: switch on Integers

```
#include <iostream> /* File: switch-find-comp2011-instructor.cpp */
using namespace std;

int main()                  // To determine your instructor
{
    cout << "Enter the COMP2011 section number to find its instructor: ";
    int section;            // COMP2011 section number: should be 1, 2, 3, or 4
    cin >> section;         // Input COMP2011 section number

    switch (section)
    {
        case 1:
            cout << "Sergey Brin" << endl; break;
        case 2:
            cout << "Bill Gates" << endl; break;
        case 3:
            cout << "Steve Jobs" << endl; break;
        case 4:
            cout << "Jeff Bezos" << endl; break;
        default:
            cerr << "Error: Invalid lecture section " << section << endl;
            break;
    }

    return 0;
}
```

Example: switch on Characters

```
#include <iostream> /* File: switch-char-bloodtype.cpp */
using namespace std;

int main()                  // To find out who may give you blood
{
    cout << "Enter your blood type (put 'C' for blood type AB): ";
    char bloodtype; cin >> bloodtype;

    switch (bloodtype)
    {
        case 'A':
            cout << "Your donor must be of blood type: O or A\n";
            break;
        case 'B':
            cout << "Your donor must be of blood type: O or B\n";
            break;
        case 'C':
            cout << "Your donor must be of blood type: O, A, B, or AB\n";
            break;
        case 'O':
            cout << "Your donor must be of blood type: O";
            break;
        default:            // To catch errors
            cerr << "Error: " << bloodtype << " is not a valid blood type!\n";
            break;
    }
    return 0;
}
```

Example: switch with Sharing Cases

```
#include <iostream> /* File: switch-int-grade.cpp */
using namespace std;

int main()                  // To determine your grade (fictitious)
{
    char grade;             // Letter grade
    int mark;               // Numerical mark between 0 and 100
    cin >> mark;

    switch (mark/10)
    {
        case 10:            // Several cases may share the same action
        case 9:
            grade = 'A'; break; // If mark >= 90
        case 8: case 7: case 6: // May write several cases on 1 line
            grade = 'B'; break; // If 90 > mark >= 60
        case 5:
        case 4:
        case 3:
        case 2:
            grade = 'C'; break; // If 60 > mark >= 20
        case 1:
            grade = 'D'; break; // If 20 > mark >= 10
        default:
            grade = 'F'; break;
    }

    cout << "Your letter grade is " << grade << endl;
    return 0;
}
```

Example: switch vs. if-else-if

```
#include <iostream> /* File: if-elseif-grade.cpp */
using namespace std;

int main()                  /* To determine your grade (fictitious) */
{
    char grade;             // Letter grade
    int mark;               // Numerical mark between 0 and 100
    cin >> mark;

    if (mark >= 90)
        grade = 'A';        // mark >= 90
    else if (mark >= 60)
        grade = 'B';        // 90 > mark >= 60
    else if (mark >= 20)
        grade = 'C';        // 60 > mark >= 20
    else if (mark >= 10)
        grade = 'D';        // 20 > mark >= 10
    else
       grade = 'F';         // 10 > mark

    cout << "Your letter grade is " << grade << endl;
    return 0;
}
```

Remarks on switch

- The expression for switch must evaluate to an integral value (integer, char, bool in C++).
- NO 2 cases may have the same value.
- On the other hand, several cases may share the same action statements.
- When a case constant is matched, the statements associated with the case are executed until either
  - a break statement.
  - a return statement.
  - the end of the switch statement.
- Difference between a switch statement and a if-else-if statement:
  - switch statement can only test for equality of the value of one quantity.
  - each expression of the if-else-if statement may test the truth value of different quantities or concepts.

Example: Give me a break

```
#include <iostream> /* File: switch-no-break.cpp */
using namespace std;

int main()                  // To determine your grade (fictitious)
{
    char grade;             // Letter grade
    int mark;               // Numerical mark between 0 and 100
    cin >> mark;

    /* What happens if you forget to break? What is the output? */
    switch (mark/10)
    {
        case 10: case 9:
            cout << "Your grade is A" << endl;
        case 8: case 7: case 6:
            cout << "Your grade is B" << endl;
        case 5: case 4: case 3: case 2:
            cout << "Your grade is C" << endl;
        case 1:
            cout << "Your grade is D" << endl;
        default:
            cout << "Your grade is F" << endl;
    }

    return 0;
}
```

New Data Types with enum

- One way to define a new data type is to use the keyword enum.

Syntax: enum Declaration

enum new-datatype { identifier1 [=value1], identifier2 [=value2], ··· };

Example

```
enum weekday { MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY,
               SATURDAY, SUNDAY };            // 0,1,2,3,4,5,6

enum primary_color { RED = 1, GREEN, BLUE }; // 1,2,3

enum bloodtype { A, B, AB = 10, O };          // 0,1,10,11
```

User-defined enum Type

- An enumeration is a type that can hold a finite set of symbolic objects.
- The symbolic (meaningful) names of these objects follow the same rule as identifier names.
- The symbolic names make your program easier to read/understand.
- Internally, these objects are represented as integers.
- By default, the first object is given the value zero, then each subsequent object is assigned a value one greater than the previous object's value.
- The integral values of the enumerated objects may be assigned other integral values by the programmer.
- Thus, the objects of an enum type act like named integer constants.

Example: enum with switch

```
#include <iostream> /* File: enum-shapes.cpp */
using namespace std;

int main()
{
    enum shapes { TEXT, LINE, RECT, CIRCLE };
    cout << "supported shapes: "
         << " TEXT = " << TEXT << " LINE = " << LINE
         << " RECT = " << RECT << " CIRCLE = " << CIRCLE << endl;
    int myshape; // Why the type of myshape is not shape?
    cin >> myshape;

    switch (myshape)
    {
        case TEXT:
            cout << "Call a function to print text" << endl; break;
        case LINE:
            cout << "Call a function to draw a line" << endl; break;
        case RECT:
            cout << "Call a function to draw a rectangle" << endl; break;
        case CIRCLE:
            cout << "Call a function to draw a circle" << endl; break;
        default:
            cerr << "Error: Unsupported shape" << endl; break;
    }

    return 0;
}
```

Part III

Further Readings and Examples

Quiz: Logical Operations

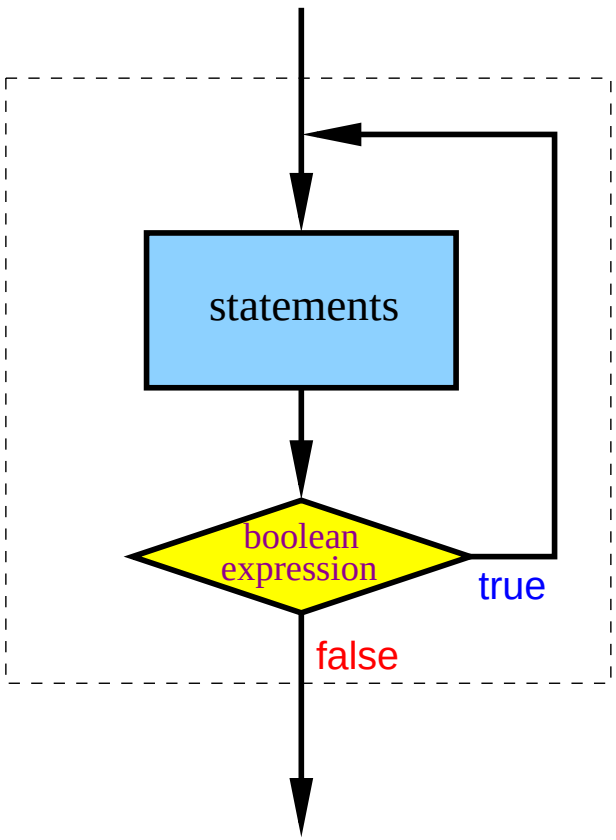
What is the value of each of the following boolean expressions:

- $4 == 5$
- $x > 0$ && $x < 10$ /* if int x = 5 */
- $5 * 15 + 4 == 13$ && $12 < 19$ || $!\text{false} == 5 < 24$
- true && false || true
- x /* if int x = 5 */
- $x++ == 6$ /* if int x = 5 */
- $x = 9$
- $x == 3 == 4$ /* assume that x is an int */

do-while Loop (Statement)

Syntax: do-while Statement

do { <stmts> } while (<bool-exp>);



- Again, like the while statement, <stmts> will be repeated as long as the value of <bool-exp> is true.
- However, unlike the the while statement, the <bool-exp> is evaluated after <stmts> at the bottom of do-while statement.
- That means, <stmts> in do-while loop will be executed at least once, whereas <stmts> in while loop may not be executed at all.

Example: Factorial using do-while Loop

```
#include <iostream> /* File: do-factorial.cpp */
using namespace std; // Compute x! = x(x-1)(x-2)...1; x is non -ve

int main()
{
    int factorial = 1, number;
    cout << "Enter a non-negative integer: ";
    cin >> number;

    if (number > 0)
    {
        do
        {
            factorial *= number; // Same as: factorial = factorial*number
            --number;            // Same as: number = number-1
        } while (number > 1);
    }

    cout << factorial << endl;
    return 0;
}
```

Which Loop to Use?

for loop :

- When you know how to specify the required number of iterations.
- When the counter variable is also needed for computation inside the loop.
- e.g. To compute sums, products, and to count.

while loop :

- You want to repeat an action but do not know exactly how many times it will be repeated.
- The number of iterations is determined by a boolean condition. e.g.

```
while (cin >> x) { ... }
```

do-while loop :

- The associated actions have to be executed at least once.
- Otherwise, do-while and while are used in similar situations.

Common Loop Errors

What is the error in each of the following cases?

- Case 1:

```
int sum;
while (cin >> x)
    sum += x;
```

- Case 2:

```
int j;
while (j < 10)
{
    cout << "hello again!" << endl;
    j++;
}
```

- Case 3:

```
int j = 0;
while (j < 10);
{
    cout << "hello again!" << endl;
    j++;
}
```

More enum Example: Mixing Colors

```
#include <iostream> /* File: enum-colors.cpp */
using namespace std;

int main()
{   // Declare color variables immediately after the enum definition
    enum color { RED, GREEN, BLUE, YELLOW, CYAN, PURPLE } x, y;
    int xint, yint;     // Input variables for the color variables

    cin >> xint >> yint;
    x = static_cast<color>(xint); // Convert an int to a color quantity
    y = static_cast<color>(yint); // Convert an int to a color quantity

    if ( (x == RED && y == GREEN) || (y == RED && x == GREEN) )
        cout << YELLOW << endl;

    else if ( (x == RED && y == BLUE) || (y == RED && x == BLUE) )
        cout << PURPLE << endl;

    else if ( (x == GREEN && y == BLUE) || (y == GREEN && x == BLUE) )
        cout << CYAN << endl;

    else
        cerr << "Error: only support mixing RED/GREEN/BLUE!" << endl;

    return 0;
} // Check what is really printed out
```